Lab 1.b — Fixing Bugs, Test-First

AI-assisted bug fixing with Claude and the FSL playbook

FSL AI Training · TaskBoard lab

TODAY

Two hours, hands-on

15 min	**Recap + the method**	Why test-first, the loop, and how to prompt Claude
25 min	**Demo — Bug #5**	Case-sensitive search, fixed live
40 min	**Your turn — Bug #4**	"Completed this week" always 0
20 min	**Review**	We compare a few of your fixes
15 min	**Feature**	Same rhythm, a new capability
5 min	**Wrap**	Checklist + where to get help

SETUP

Install your tooling

One-time, before lab day — macOS via Homebrew, Windows via winget.

Tool	macOS	Windows
Git	`xcode-select --install` · or brew install git	`winget install Git.Git`
nvm (Node manager)	`brew install nvm`	`winget install CoreyButler.NVMforWindows`
Node LTS ≥ 20.19 + npm	`nvm install 20 && nvm use`	`nvm install 20 && nvm use 20`
GitHub CLI	`brew install gh` → gh auth login	`winget install GitHub.cli` → gh auth login
Claude Code	`curl -fsSL https://claude.ai/install.sh | bash`	`irm https://claude.ai/install.ps1 | iex`
VS Code (or your editor)	`brew install --cask visual-studio-code`	`winget install Microsoft.VisualStudioCode`

ACCOUNTS GitHub account + Claude Code seat (company-provisioned) — confirm both logins before lab day.
VERIFY `git --version · node --version (≥ 20.19) · gh auth status · claude --version`

SETUP

Set up the training repo

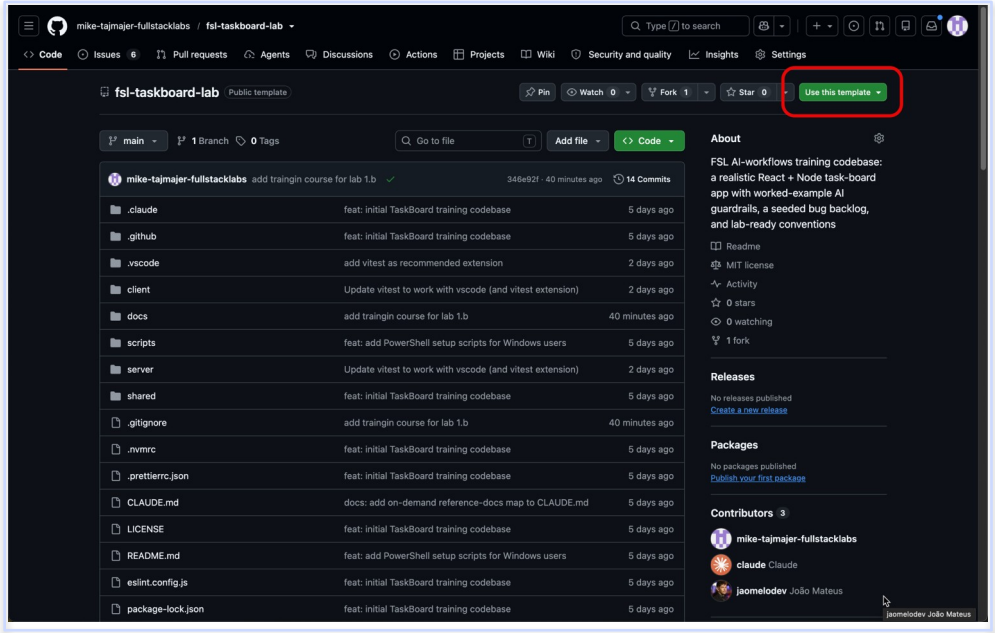
github.com/mike-tajmajer-fullstacklabs/fsl-taskboard-lab

1. On that repo, click Use this template → Create a new repository (green button, top-right — circled). Your own copy, not a fork.

2. Clone it, then from the repo root seed the backlog: ./scripts/setup-repo.sh (Windows: .\scripts\setup-repo.ps1). Creates the lab labels + issues — gh login done on the previous slide.

3. Install & run: npm install · npm run reset-db · npm run dev (client :5173, API :3001). Node LTS ≥ 20.19 — nvm use reads .nvmrc.

4. Confirm npm test is green, then pick a lab-1 issue.



Start from "Use this template" — not Fork or Clone.

RECAP — LAB 1.A

You already have what you need

- This repo already ships a strong CLAUDE.md — Claude knows the stack, conventions, and commands.
- You've met Plan Mode — plan first, review the plan, then let Claude build.
- Today we point those at real bugs in the backlog and ship real PRs.

THE SHIFT

Lab 1.a: set up context.
Lab 1.b: use that context to fix defects the disciplined way — test-first, verified, reviewed.

WHY

A bug is a missing test

A bug means some behavior was never pinned down by a test. So we write that test first — then fixing it becomes provable, not hopeful.

REPRODUCE RELIABLY

- Turn "it's broken" into a repeatable, red test
- No more "works on my machine"

A PRECISE TARGET FOR CLAUDE

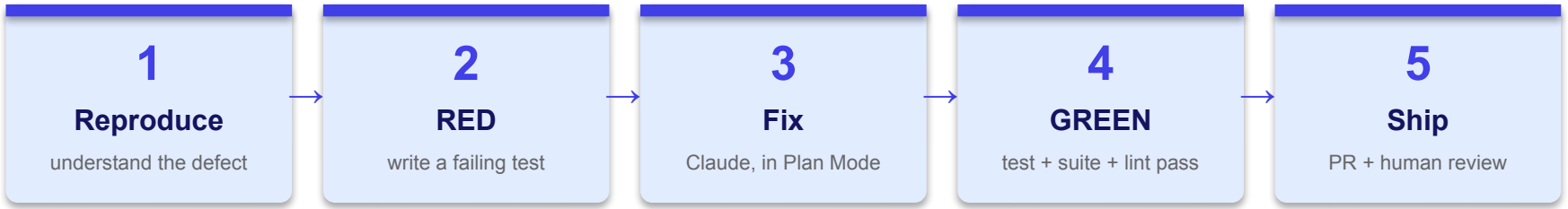
- The failing test tells Claude exactly what "fixed" means
- Less guessing, tighter fixes

PROOF + NO REGRESSIONS

- Green test = objectively fixed
- The test stays forever, so it can't silently break again

THE METHOD

The test-driven fix loop



Same five steps every time — for the demo, your exercise, and (later) a feature.

Habits that keep fixes safe

- Feed Claude the failing test + let CLAUDE.md carry the conventions.
- Plan Mode first — review the plan before any code changes.
- Don't accept blindly — you own the diff; read it and course-correct.
- Keep the guardrails — never touch seed.json (a hook blocks it).
- Green means green — npm test, lint, and typecheck all pass.
- Ship it right — branch fix/<slug>, PR "Closes #N", a human reviewer, CI green.

PROMPTING

RTCE and the FSL pattern: one shape

The two courses teach it under different names — the same four ideas. Read across:

RTCE (Ag. Eng.)	FSL / Claude Code	What you actually write
Role	→ part of Context	"You're in the TaskBoard Express API" — usually already in CLAUDE.md
Task	= Task	"Add a failing test for case-insensitive search"
Context	= Context	"@server/src/services/todos.service.ts; use makeTestDb"
Expectations	→ Constraints + Verify	"don't refactor; then run npm test -w server"

Why the FSL grouping wins: it pulls Verify out of "Expectations" as its own step — so every prompt says HOW Claude proves it's done ("run the tests"), not just what you hoped for. That's the habit that makes a fix provable.

Where it lives: CLAUDE.md holds Role + Context (shared) · your prompt holds Task + Constraints + Verify.

PROMPTING

Three habits that make it work

DESCRIBE SYMPTOMS, NOT SOLUTIONS

- Say what you observe, not your diagnosis.
- "Search 'FIRST' finds nothing" — let Claude find the cause.

POINT AT REAL FILES WITH @

- Show, don't describe.
- "@server/src/services/todos.service.ts" beats a paraphrase.

END WITH "RUN THE TESTS"

- Make Claude verify its own work.
- "…then run npm test -w server and fix any failures."

These three turn the four-part shape into prompts that land the first time.

PROMPTING

Same goal, four ways to ask

Goal: a failing test that proves search is case-sensitive. All four are good — pick what fits what you already know.

EXPLICIT (CONTEXT · TASK · CONSTRAINTS · VERIFY)

In @server/src/services/todos.service.ts, list() search is case-sensitive. Add a Vitest test (todos.service.test.ts, makeTestDb) that searches q:'first' and expects todo_a; run npm test -w server and show it fail.

SYMPTOM-FIRST

Searching 'FIRST' finds nothing but 'First' works. Investigate list() in @server/src/services/todos.service.ts, then write a failing test that captures the correct case-insensitive behavior.

REFERENCE-DRIVEN (SHOW, DON'T DESCRIBE)

Follow the makeTestDb pattern in @server/src/services/todos.service.test.ts. Add a search test proving search should match regardless of case; run npm test -w server.

PLAN-FIRST

Before any code: read list() and tell me why 'first' misses 'First fixture todo', and the failing test you'd add. I'll confirm, then you write it.

All four produce the same red test — different roads for how much you already know.

Demo — Bug #5

"Todo search is case-sensitive" · watch the loop end to end

REPRODUCE

What's actually wrong

- Symptom: searching "FIRST" finds nothing, but "First" does.
- Lives server-side: todos.service.ts → list().
- Root cause: includes() is case-sensitive and nothing is normalized.

THE CODE TODAY

```
if (query.q) {
  const q = query.q;
  todos = todos.filter((t) =>
    t.title.includes(q) ||
    (t.notes ?? '').includes(q));
}
```

First we make the bug repeatable as a test — then the fix is provable.

RED

Write the failing test first

```
// server/src/services/todos.service.test.ts
import { afterEach, beforeEach, describe,
  expect, it } from 'vitest';
import { todosService } from './todos.service';
import { makeTestDb } from '../testing/helpers';

describe('list — search', () => {
  let db: ReturnType<typeof makeTestDb>;
  beforeEach(() => { db = makeTestDb(); });
  afterEach(() => db.cleanup());
  it('matches case-insensitively', () => {
    const { todos } = todosService.list({
      q: 'first', sort: 'createdAt',
      page: 1, pageSize: 20 });
    expect(todos.map(t => t.id))
      .toContain('todo_a');
  }); });
```

WHY IT'S RED

"First fixture todo" won't match lowercase "first" today — so the test fails. Now you have a precise target.

TRY — EXPLICIT

Add a Vitest test (makeTestDb) that searches q:'first', expects todo_a; run npm test -w server.

OR — SYMPTOM-FIRST

'FIRST' finds nothing but 'First' works — investigate list() and write a failing test.

GREEN

The fix — lowercase both sides

```
// todos.service.ts — list()
- const q = query.q;
+ const q = query.q.toLowerCase();
  todos = todos.filter((t) =>
-   t.title.includes(q) ||
-   (t.notes ?? '').includes(q));
+   t.title.toLowerCase().includes(q) ||
+   (t.notes ?? '').toLowerCase()
+     .includes(q));
```

VERIFY

- npm test -w server
- npm run lint
- npm run typecheck
- All green → the bug is fixed.

MINIMAL FIX

Make the minimal change to pass the failing test — don't refactor anything else. Then run npm test -w server.

PLAN MODE

In Plan Mode: propose the smallest fix to list(); show me the plan before editing.

SHIP IT

From green test to merged PR

1. Branch: git checkout -b fix/case-insensitive-search
2. Commit (Conventional): fix: make todo search case-insensitive
3. Open a PR — the template links the issue with "Closes #5".
4. CI runs lint + typecheck + tests; it must be green.
5. Get one human reviewer before merge.

DONE MEANS

- New test added + passing
- Full suite green in CI
- PR links #5
- Reviewed by a teammate

Your turn — Bug #4

"Stats: Completed this week always shows 0" · same loop, you drive

THE TASK

Diagnose and fix it, test-first

- Symptom: the dashboard's "Completed this week" stat stays at 0 — even after you complete todos.
- Where to look: server/src/services/stats.service.ts → summary().
- You diagnose the root cause — that's the exercise. Follow the same five-step loop.

THE LOOP

Reproduce → RED → Fix (Plan Mode) → GREEN → PR.

Start by making it fail in a test. The fix comes second.

HINTS & ACCEPTANCE

Get unstuck without the answer

```
// stats.service.test.ts  (create it —
//  no stats test exists yet)
beforeEach(() => { db = makeTestDb(); });
afterEach(() => db.cleanup());
it('counts todos done in last 7 days',
   () => {
  todosService.complete('todo_a');
  // ^ stamps completedAt = now
  // TODO: assert summary()
  //   .completedThisWeek is 1
});
```

REMEMBER

- Complete a todo now — the fixture's done one is old, so it won't count this week.
- Watch it fail, then fix stats.service.ts.
- PR "Closes #4" + a reviewer.

DIAGNOSE (SYMPTOM-FIRST)

"Completed this week" stays 0 after I complete todos — investigate summary() in @server/src/services/stats.service.ts. Don't fix yet.

WRITE THE TEST FIRST

Create stats.service.test.ts (makeTestDb): complete a todo, then assert summary().completedThisWeek — make it fail first.

REVIEW

Let's compare a few fixes

- Did your test fail first? If it passed immediately, it wasn't really testing the bug.
- What was the root cause — and how did your test capture it?
- Did Claude's first attempt pass review, or did you course-correct? What did you catch?
- Did anything else go red? A good fix leaves the rest of the suite green.
- Compare two diffs on screen — same bug, different tests and fixes. What do we prefer, and why?

Features

Same rhythm — a different starting point

BUG VS. FEATURE

What changes, what stays

BUG FIX

- Behavior already exists — but it's wrong.
- The test reproduces the defect, then pins the correction.

FEATURE

- Behavior doesn't exist yet.
- The test specifies the new behavior — it fails until you build it.

Same rhythm either way: write the test first → let it drive the change with Claude → verify green → PR + review. The test drives; only the starting point differs.

FEATURE EXAMPLE

Sort todos by due date

1. Spec it in the type: add 'dueDate' to TodoQuery.sort (shared/src/types.ts).
2. Test first: list({ sort: 'dueDate' }) returns due-date order, undated last.
3. Implement the sort branch in todos.service.ts (createdAt only today).
4. Add a sort selector in TodoFilterBar.tsx.
5. Verify green, then PR "Closes #" the feature issue.

THE TEST LEADS

```
it('sorts by due date,
   undated last', () => {
  const { todos } =
    todosService.list({
      sort: 'dueDate' });
  expect(todos[0].id)
    .toBe('todo_b');
});
```

INTERVIEW / PLAN-FIRST

Before coding, interview me on edge cases (no due date, direction) and propose the plan across the shared type, service, and filter bar.

EXPLICIT TEST-FIRST

Add 'dueDate' to TodoQuery.sort in @shared/src/types.ts, then a failing test that list({sort:'dueDate'}) orders by due date (undated last) before implementing.

REFERENCE

Prompt starters — keep these handy

Reproduce / diagnose	Run the failing test with npx vitest run <file>; then trace the code and tell me the root cause before changing anything.
Write the failing test	Add a Vitest test in the co-located *.test.ts (makeTestDb) that captures the correct behavior; run it and show it red.
Fix — minimal	Make the minimal change to pass the failing test. Don't refactor anything else. Then run npm test -w server.
Fix — Plan Mode	In Plan Mode: propose the smallest fix; show me the plan before editing.
Verify	Run npm test -w server, then the full suite + lint + typecheck. Add a comment on the test noting the bug it prevents.

Swap in your paths. Describe symptoms · @-reference real files · end with "run the tests."

The loop is the takeaway

Reproduce → RED → Fix (Plan Mode) → GREEN → PR & review.

Ship checklist: a new test that failed first and now passes · full suite, lint & typecheck green · branch fix/<slug> · PR "Closes #N" · one human reviewer · CI green.

Questions & support: your team's AI-training channel on Slack — or ask your instructor.